LAB 3 – gRPC & Microservices Architecture

Assignment Requirement

Continue developing the Learning Management System (LMS) from Lab 2.

Students must refactor the monolithic application into a Microservices Architecture and implement service-to-service communication using gRPC.

The following requirements from previous labs must be maintained:

- Clean Architecture
- Dependency Injection
- JWT Authentication
- Docker Deployment
- Swagger/OpenAPI

---

1. Microservices Architecture

The system must be separated into at least three independent services.

Identity Service

Responsibilities:

- Authentication
- Authorization
- JWT Generation

Student Service

Responsibilities:

- Student Management
- Student Information

Course Service

Responsibilities:

- Course Management
- Enrollment Management

Requirements:

- Each service must be an independent ASP.NET Core Web API project.
- Each service must have its own database.
- Direct access to another service’s database is prohibited.

2. gRPC Communication

Students must implement gRPC communication between services.

Required Scenario

Course Service must retrieve student information from Student Service through gRPC.

Example:

```
Course Service
      |
     gRPC
      |
Student Service
```

Requirements:

- Implement at least one gRPC Server.
- Implement at least one gRPC Client.
- Use Protocol Buffers (.proto).
- Use strongly typed gRPC clients.

---

3. API Gateway

Implement an API Gateway using one of the following:

- YARP Reverse Proxy
- Ocelot

Requirements:

- Route requests to the appropriate service.
- Validate JWT tokens before forwarding requests.

Example:

```
/api/auth/*      → Identity Service
/api/students/*  → Student Service
/api/courses/*   → Course Service
```

---

4. Authentication & Authorization

Continue using JWT authentication from Lab 2.

Requirements:

- Login API
- JWT Validation
- Refresh Token
- Protected APIs
- Role-based Authorization

Example:

```
[Authorize]

[Authorize(Roles = "Admin")]
```

---

5. Service-to-Service Business Flow

Implement the following business process.

Enroll Student into Course

Processing flow:

```
Client
   |
API Gateway
   |
Course Service
   |
gRPC
   |
Student Service
```

Requirements:

- Verify student existence through gRPC.
- Enrollment is allowed only for valid students.

---

6. Docker Deployment

Deploy the entire system using Docker Compose.

Requirements:

- Dockerfile for each service.
- docker-compose.yml for the complete solution.

Minimum containers:

```
api-gateway

identity-service
student-service
course-service

identity-db
student-db
course-db
```

---

7. Logging

Implement logging using Serilog.

The system must log:

- Request Path
- HTTP Method
- Status Code
- Execution Time

---

8. Swagger Documentation

Each service must provide Swagger/OpenAPI documentation.

Requirements:

- JWT Authentication support
- Testing of protected endpoints through Swagger

---

9. Testing

Students must demonstrate the following scenarios:

Test Case	Expected Result
Login	JWT token generated
Protected API Access	Success
Unauthorized Request	HTTP 401
gRPC Communication	Data returned successfully

Test Case	Expected Result
Course Enrollment	Enrollment completed

Deliverables

Students must submit:

- Source Code
- Dockerfile(s)
- docker-compose.yml
- Proto Files (.proto)
- Postman Collection
- Architecture Report (2–3 pages) including:
- Service decomposition
- Database design
- API Gateway configuration
- gRPC communication flow

Bonus (+10%)

- RabbitMQ Integration
- Redis Cache
- OpenTelemetry Distributed Tracing
- Polly Circuit Breaker for gRPC Clients